

Farmer Marketplace: A MERN Stack Platform for Direct Farm-to-Customer Sales

Final Project Report

1. Cover Page

Project Title: Farmer Marketplace: A Full-Stack MERN Web Application for Direct Farm-to-Customer Trade

Student Names & Roll Numbers:

S.No	Name	Roll Number
1	[Student Name 1]	[Roll No.]
2	[Student Name 2]	[Roll No.]
3	[Student Name 3]	[Roll No.]

Department: [Department of Computer Science and Engineering / Information Technology]

College: [College Name, City]

Guide Name: [Prof. / Dr. Guide Name]

Academic Year: [20XX - 20XX]

2. Abstract & Introduction

Abstract

Farmer Marketplace is a full-stack web application built on the MERN stack (MongoDB, Express.js, React, Node.js). It lets farmers list and sell fresh produce directly to customers, removing the middlemen who normally take a large share of the price. Customers can search, filter, and order produce, while an administrator approves farmers and moderates listings. The system uses JWT authentication, bcrypt password hashing, and role-based access control for three roles (farmer, customer, admin). Orders are placed with Cash on Delivery, stock is reduced automatically, and every order moves through a clear status flow (pending, accepted, shipped, delivered, or cancelled). A

farmer dashboard shows revenue and top-selling produce using Recharts. The backend can fall back to an in-memory MongoDB with seeded demo data, so the project runs out of the box without a separate database installation.

Project Overview

The platform has three parts: a React single-page frontend, a Node/Express REST API, and a MongoDB database. Farmers manage products and incoming orders. Customers browse the catalog, use a cart, check out, and track orders. Admins manage users and products and view platform statistics.

Problem Background

In traditional agricultural supply chains, produce passes through several intermediaries before it reaches the buyer. Farmers receive a small fraction of the final price, while customers pay more for produce that may be less fresh. Small farmers often lack the digital tools and market access to sell directly, and customers have little visibility into where their food comes from.

Proposed Solution

A web marketplace where farmers register, set their own prices, and publish produce with images, quantity, and location. Customers can discover fresh produce near them and order it directly. Admin approval of farmers and moderation of listings build trust on the platform.

Purpose of the Project

- Give farmers a direct sales channel and fairer pricing.
 - Give customers fresher produce at transparent prices.
 - Demonstrate a complete MERN application with REST APIs, authentication, role-based access, and data visualization.
-

3. Problem Statement, Objectives & Scope

Problem Statement

Farmers lack a simple, low-cost digital platform to sell directly to consumers, so they depend on intermediaries who reduce their income. Customers, in turn, cannot easily search for, compare, and order fresh local produce. There is a need for a secure, role-based web system that connects the two groups and manages listings, stock, and orders end to end.

Project Objectives

1. Build a full-stack application using the MERN stack.
2. Implement CRUD operations in MongoDB for users, products, orders, and reviews.
3. Design and expose REST APIs for authentication, products, orders, reviews, and admin functions.
4. Develop a React frontend with protected routes for each role.
5. Implement JWT authentication and role-based authorization (farmer, customer, admin).
6. Automate stock reduction on order placement and reject orders when stock is insufficient.
7. Provide search, filtering, and sorting of products by keyword, category, price, and location.
8. Present results clearly through dashboards, charts, order status badges, and a clean product catalog.

Project Scope

In scope: user registration and login; farmer approval; product CRUD with image upload; catalog search and filters; cart and Cash on Delivery checkout; order tracking and status updates; reviews and ratings; farmer sales dashboard; admin moderation and statistics.

Out of scope (in this version): online payment gateways, real-time delivery tracking, mobile native apps, and multi-language support.

Target Users

User	Description
Farmers	Small and medium farmers who want to sell produce directly
Customers	Households and buyers looking for fresh, local produce
Administrators	Platform managers who approve farmers and moderate content

4. Features & Functional Requirements

Main Features

- Role-based system with farmer, customer, and admin accounts.
- Product catalog with search, category, location, and price filters, plus sorting.
- Shopping cart saved in localStorage with stock limit warnings.
- Cash on Delivery checkout with address and contact details.
- Order lifecycle: pending, accepted, shipped, delivered, or cancelled.
- Automatic stock reduction after an order is placed.
- Farmer dashboard with revenue, order, and product statistics and charts.
- Admin panel for user approval, deletion, product moderation, and analytics.
- Product reviews and ratings.
- Quick demo login buttons for fast evaluation.

User Functionalities

Farmer

- Register and wait for admin approval.
- Add, edit, and delete products (name, category, price, unit, quantity, description, location, image).
- View incoming orders and accept, reject, ship, or deliver them.
- View sales statistics and charts.

Customer

- Register and log in.
- Search and filter products; view product details and reviews.
- Add items to the cart, check out, and view order history and status.
- Post ratings and reviews.

Admin

- View all users, approve or block farmers, and delete users.
- View all listings and remove inappropriate products.
- View totals for users, farmers, products, and sales volume.

CRUD Operations

Entity	Create	Read	Update	Delete
User	Register	Get profile, admin user list	Admin approves or blocks	Admin deletes
Product	Farmer adds product	Catalog and detail views	Owner edits	Owner or admin deletes
Order	Customer places order	Customer, farmer, and admin views	Farmer updates status	Cancelled (status change)
Review	Customer adds review	Product detail page	N/A	N/A

Authentication & Validation

- Passwords are hashed with **bcrypt** before storage.
- A **JWT** is issued at login and stored on the client; an Axios interceptor attaches it to every request.
- The `protect` middleware verifies the token; `authorize('farmer' | 'customer' | 'admin')` checks the role.
- Unapproved farmers cannot publish products.
- Server-side validation covers required fields, valid email format, positive price and quantity, and sufficient stock at checkout.
- A global error handler returns clear, consistent error messages to the client.

System Requirements

Hardware (minimum): dual-core processor, 4 GB RAM, 1 GB free disk space.

Software: Windows, Linux, or macOS; Node.js 18 or later; npm; a modern browser (Chrome, Edge, Firefox); MongoDB (local or Atlas), or the built-in in-memory fallback; VS Code and Postman for development and testing.

5. Algorithms & Methodology

System Workflow

1. A user registers as a farmer or customer. Farmers start unapproved until an admin approves them.

2. The user logs in and receives a JWT; the frontend redirects based on role.
3. An approved farmer lists products with stock and price.
4. A customer searches and filters the catalog, then adds items to the cart.
5. At checkout the customer enters an address and places a COD order.
6. The server validates stock, creates the order, and reduces the product quantities.
7. The farmer accepts the order and moves it through shipped and delivered.
8. The customer tracks the status and may leave a review.
9. The admin monitors users, products, and platform statistics.

Algorithms Used

1. Password hashing (bcrypt): a salted hash is generated on registration, and login compares the entered password with the stored hash.

2. JWT authentication and authorization:

```
protect(req):
    token = read Authorization header
    payload = verify(token, SECRET)
    req.user = find User by payload.id
    if no user: reject (401)

authorize(roles):
    if req.user.role not in roles: reject (403)
```

3. Product search and filtering: a MongoDB query is built dynamically from the request parameters. Keyword search uses a case-insensitive regex on name and description, category is matched exactly, location uses string matching, and price range and sort order are applied before pagination.

4. Order placement with stock validation:

```
placeOrder(items):
    total = 0
    for each item:
        product = find Product by id
        if product.quantity < item.qty: reject with friendly warning
        total += product.price * item.qty
    for each item:
        product.quantity -= item.qty ; save
    create Order(status = pending, total)
```

5. Order status state machine: allowed transitions are pending -> accepted -> shipped -> delivered, and pending/accepted -> cancelled. Only the owning farmer can update the status of an order.

6. Dashboard aggregation: MongoDB aggregation pipelines group orders by month for revenue and by product for top-selling produce, which feed the Recharts graphs.

Development Methodology

The project followed an **iterative, incremental (Agile-style)** approach, building and testing one module at a time:

1. Setup: initialize server and client, connect MongoDB, test with Postman.
2. Authentication: register, login, JWT, role middleware.
3. Product CRUD: the core module, built and tested first.
4. Frontend authentication and product pages.
5. Cart and orders with stock reduction, then farmer order management.
6. Admin panel and dashboard statistics.
7. Polish: validation, error messages, loading states, responsive design.

Data Flow

Level 0 (context): Users (farmer, customer, admin) interact with the Farmer Marketplace system, which stores and retrieves data from MongoDB.

Level 1:

1. The React client sends an HTTP request through Axios, with the JWT attached.
2. An Express route receives it and passes it through `protect` and `authorize`.
3. The controller applies business logic and uses a Mongoose model.
4. MongoDB stores or returns the data.
5. The controller returns a JSON response, and React updates the UI.

Database Design

Collection	Key Fields
User	name, email, password (hashed), role, phone, location, isApproved
Product	name, category, price, quantity, unit, description, image, location, farmer (ref User), createdAt

Collection	Key Fields
Order	customer (ref), items [{product, farmer, qty, price}], totalAmount, status, address, createdAt
Review	product (ref), customer (ref), rating, comment

6. Technologies & Implementation

Frontend Technologies

Technology	Purpose
React (Vite)	Component-based single-page application
React Router	Routing and role-based protected routes
Context API	Global state for authentication and cart
Axios	HTTP client with a JWT interceptor
Tailwind CSS	Responsive styling
Recharts	Sales and top-product charts

Backend Technologies

Technology	Purpose
Node.js	Server runtime
Express.js	REST API framework
JSON Web Token	Stateless authentication
bcrypt	Password hashing
Multer	Product image upload
dotenv	Environment configuration

Database

MongoDB with Mongoose ODM. The backend connects to a local MongoDB or Atlas using the connection string in `.env`. If none is available, it automatically starts `mongodb-memory-server` and runs a seed script, so the application works instantly on any machine. The seed script creates demo farmers, products, orders, and an admin account.

Tools & APIs

- **VS Code** for development, **Postman** for API testing, **Git/GitHub** for version control.
- **MongoDB Atlas** (optional) for a cloud database.
- Deployment options: Render or Railway for the backend, Vercel or Netlify for the frontend.

Implementation Details

Project structure

```

farmer-marketplace/
├── server/
│   ├── config/db.js
│   ├── models/           (User, Product, Order, Review)
│   ├── middleware/       (auth.js, upload.js, error.js)
│   ├── controllers/      (auth, product, order, admin, review)
│   ├── routes/
│   ├── seed.js
│   ├── uploads/
│   └── server.js
└── client/
    ├── src/
    │   ├── api/axios.js
    │   ├── context/      (AuthContext, CartContext)
    │   ├── components/   (Navbar, Footer, ProductCard, ProtectedRoute,
StatusBadge, StatsCard)
    │   ├── pages/        (Home, Login, Register, ProductList, ProductDetail,
Cart, Checkout,
    │   │                  MyOrders, FarmerDashboard, FarmerProducts,
FarmerAddEditProduct,
    │   │                  FarmerOrders, AdminPanel)
    │   └── App.jsx

```

REST API summary

Area	Endpoints
Auth	POST /api/auth/register , POST /api/auth/login , GET /api/auth/me
Products	GET /api/products (? search=&category=&location=), GET /api/products/:id , POST /api/products (farmer), PUT /api/products/:id (owner), DELETE /api/products/:id (owner or admin)
Orders	POST /api/orders (customer), GET /api/orders/my (customer), GET /api/orders/farmer (farmer), PUT /api/orders/:id/status (farmer)
Reviews	POST /api/reviews , GET /api/reviews/:productId
Admin	GET /api/admin/users , PUT /api/admin/users/:id/approve , DELETE /api/admin/products/:id , GET /api/admin/stats

Key implementation notes

- `AuthContext` stores the user and token and exposes login, logout, and role helpers; `ProtectedRoute` blocks pages for the wrong role.
- `CartContext` keeps cart items in `localStorage` and limits quantities to available stock.
- `StatusBadge` shows each order status in a different color for quick reading.
- Products can be sorted and paginated on the listing page.
- The login page offers one-click demo accounts for the three roles.

7. Results, Testing & Performance

Project Results

The application delivers all planned modules: authentication, product management, catalog search and filters, cart, Cash on Delivery checkout, order tracking, farmer dashboard, reviews, and the admin panel. Farmers can sell directly, customers can order and track produce, and admins can moderate the platform. Stock updates correctly after each order, and insufficient stock is rejected with a clear message.

Test Cases

ID	Test Case	Input / Action	Expected Result	Status
TC01	Register a customer	Valid name, email, password	Account created, token returned	[Pass]
TC02	Register with an existing email	Duplicate email	Error message shown	[Pass]
TC03	Login with valid credentials	Correct email and password	JWT issued, redirect by role	[Pass]
TC04	Login with a wrong password	Incorrect password	401 error	[Pass]
TC05	Access a farmer route as a customer	Customer token on farmer API	403 Forbidden	[Pass]
TC06	Create a product	Farmer adds "Organic Honey Crisp Apples"	Product saved and visible in catalog	[Pass]
TC07	Search products	Keyword "Apples", category Fruits	Matching products only	[Pass]
TC08	Add to cart beyond stock	Quantity greater than available	Stock warning, quantity limited	[Pass]
TC09	Place an order	3 kg of apples, COD address	Order created with status pending	[Pass]
TC10	Stock deduction	Check product after TC09	Stock reduced by 3 kg	[Pass]
TC11	Order with insufficient stock	Quantity above stock	Order rejected with friendly message	[Pass]
TC12	Update order status	Farmer sets shipped, then delivered	Status updates and badge changes	[Pass]
TC13	Admin approves a farmer	Toggle approval in the admin panel	isApproved becomes true	[Pass]
TC14	Admin removes a product	Delete a listing	Product removed from catalog	[Pass]

Update the Status column with your actual test results before submission.

Testing Methods

- **Unit and API testing:** endpoints tested with Postman or a custom validation script covering registration and login, product creation and stock check, order creation and stock deduction, and the admin approval route.
- **Integration testing:** end-to-end flow from the React UI through the API to MongoDB.
- **Functional (manual) testing:** the full user journey: farmer creates a product, customer searches and orders, farmer processes the order, admin approves a new farmer.
- **Role-based security testing:** verifying that each role can reach only its permitted routes.
- **Usability and responsive testing:** checking layouts on desktop, tablet, and mobile screen sizes.

Performance Analysis

Metric	Observation
API response time	[Typically under 200 ms for catalog and order requests on a local setup; replace with measured values]
Page load	[Fast initial load with Vite build; replace with measured values]
Search and filter	Server-side filtering with pagination keeps responses small
Scalability	Stateless JWT authentication allows horizontal scaling of the API
Reliability	In-memory database fallback and seed data allow instant start-up

Performance can be improved further with database indexes on `category`, `location`, and `farmer`, image compression, and response caching.

Screenshots / Output

Insert the following screenshots, each with a caption:

1. Home page (hero section, categories, featured products)
2. Login page with demo login buttons
3. Registration page with role selector
4. Product list with search and filters

5. Product detail page with reviews
 6. Cart and checkout page
 7. My Orders page with status badges
 8. Farmer dashboard (statistic cards and Recharts graphs)
 9. Farmer product manager and add/edit product form
 10. Farmer orders page
 11. Admin panel (user approval, product moderation, statistics)
 12. Postman API test results
-

8. Conclusion, Future Scope & References

Project Conclusion

Farmer Marketplace shows how a MERN stack application can connect farmers directly with customers. The system meets its objectives: secure role-based access, complete CRUD operations, REST APIs, automatic stock management, a clear order workflow, and dashboards that present data clearly. By removing intermediaries, the platform can improve farmers' earnings and give customers access to fresh produce at fair prices. The project also gave the team practical experience in full-stack development, database design, authentication, and testing.

Future Enhancements

- Online payments through Razorpay, Stripe, or UPI.
- "Nearby farmers" search using MongoDB geospatial queries (2dsphere index) and map integration.
- Image hosting on Cloudinary instead of local storage.
- Email and SMS notifications for order updates.
- Real-time order tracking with WebSockets.
- Subscription boxes and bulk or wholesale ordering.
- Multi-language support for regional farmers.
- Mobile application using React Native.
- AI-based price suggestions and demand forecasting.
- Cloud deployment with CI/CD (Render or Railway, Vercel or Netlify, MongoDB Atlas).

References / Documentation

1. MongoDB Documentation: <https://www.mongodb.com/docs/>

2. Express.js Documentation: <https://expressjs.com/>
3. React Documentation: <https://react.dev/>
4. Node.js Documentation: <https://nodejs.org/en/docs>
5. Mongoose Documentation: <https://mongoosejs.com/docs/>
6. React Router Documentation: <https://reactrouter.com/>
7. Tailwind CSS Documentation: <https://tailwindcss.com/docs>
8. Recharts Documentation: <https://recharts.org/>
9. JSON Web Token Introduction: <https://jwt.io/introduction>
10. bcrypt (npm package): <https://www.npmjs.com/package/bcrypt>
11. Multer (npm package): <https://www.npmjs.com/package/multer>
12. Postman Learning Center: <https://learning.postman.com/>
13. Project documents: *Farmer Marketplace Implementation Plan* and *Farmer Marketplace Build Plan*.